

Нахождение всех ядерных импликант булевой функции методом SAT (без построения таблицы истинности)

1. Сведение проверки ядерности к задаче выполнимости

Пучком двоичного набора α называется

$$\Pi(f, \alpha) = \{ K_j \mid K_j(\alpha) = 1 \}.$$

По теореме 3.2 из лекций:

$$K_i \in \bigcap T_f \iff \exists \alpha : \Pi(f, \alpha) = \{K_i\}.$$

То есть K_i является ядерной импликантой тогда и только тогда, когда существует входной набор, на котором K_i равна 1, а все остальные простые импликанты равны 0.

Поэтому для каждой K_i строим формулу

$$F_i = K_i \wedge \bigwedge_{\substack{j=1 \\ j \neq i}}^m \neg K_j.$$

Тогда

$$K_i \in \bigcap T_f \iff F_i \text{ выполнима.}$$

Здесь «выполнима» означает: существует набор значений переменных, на котором формула принимает значение 1.

2. Построение F_i сразу в виде КНФ

SAT-решатели обычно принимают формулу в конъюнктивной нормальной форме (КНФ): конъюнкцию дизъюнктов, каждый из которых есть дизъюнкция литералов.

Пусть

$$K_j = l_{j1} \wedge \dots \wedge l_{jr_j},$$

где каждый литерал l_{jt} — переменная или её отрицание. По закону де Моргана

$$\neg K_j = \neg l_{j1} \vee \dots \vee \neg l_{jr_j}.$$

Поэтому построение F_i таково:

- для каждого литерала l из K_i добавляем единичный дизъюнкт (l) — это гарантирует $K_i = 1$;

- для каждого $j \neq i$ добавляем один дизъюнкт

$$(\neg l_{j1} \vee \dots \vee \neg l_{jr_j}),$$

что гарантирует $K_j = 0$.

Например, если проверяется

$$K_i = x_1 \overline{x_3},$$

то добавляются единичные дизъюнкты

$$(x_1), \quad (\overline{x_3}).$$

Если другой терм —

$$K_j = x_1 x_2 \overline{x_4},$$

то добавляется дизъюнкт

$$(\overline{x_1} \vee \overline{x_2} \vee x_4).$$

Полученная формула уже является КНФ: **никакой таблицы истинности строить не нужно, дополнительных переменных не требуется.**

3. Поиск выполняющего набора: алгоритм DPLL

Сначала выполняется **единичное распространение** (unit propagation): если дизъюнкт содержит единственный литерал, этот литерал обязан быть истинным. После присваивания переменной удаляются выполненные дизъюнкты, а из остальных удаляются ставшие ложными литералы; процесс повторяется.

Например, в

$$(x_1) \wedge (\overline{x_1} \vee x_2)$$

сначала выводится $x_1 = 1$; второй дизъюнкт превращается в (x_2) , откуда $x_2 = 1$.

В ходе распространения:

- если возник пустой дизъюнкт (все его литералы ложны) — текущая ветвь противоречива;
- если все дизъюнкты выполнены — выполняющий набор найден; неприсвоенные переменные могут получить произвольные значения;
- если остались невыполненные дизъюнкты, но единичных нет — выбирается неприсвоенная переменная, и рекурсивно проверяются оба значения 0 и 1.

Псевдокод:

DPLL(множество дизъюнктов C , текущее частичное присваивание a):

 выполнить единичное распространение в C , попутно обновляя a

 если C содержит пустой дизъюнкт:
 вернуть UNSAT

 если C пусто:
 произвольно доопределить неприсвоенные переменные в a
 вернуть SAT и набор a

 выбрать неприсвоенную переменную x из C

```

R := DPLL(C ∪ {(x)}, a)
если R = SAT:
    вернуть R

вернуть DPLL(C ∪ {(¬x)}, a)

```

Процесс работает непосредственно с дизъюнктами и частичными присваиваниями. При конфликте отсекается целая ветвь поиска, соответствующая данному частичному присваиванию, — перебирать все входные наборы под ней не требуется.

4. Алгоритм нахождения всех ядерных импликант

Литералы кодируются целыми числами со знаком: x_k — числом k , $\overline{x_k}$ — числом $-k$. Например,

$$x_1 \overline{x_3} x_5 \longleftrightarrow [1, -3, 5],$$

отрицание литерала — смена знака числа.

Полный алгоритм:

```

Вход: все простые импликанты K_1, ..., K_m
Выход: множество ядерных импликант Core

Core := ∅

для i = 1, ..., m:
    C := пустое множество дизъюнктов

    для каждого литерала l из K_i:
        добавить в C единичный дизъюнкт [l]

    для каждого j ≠ i:
        добавить в C дизъюнкт [-l : l ∈ K_j]

    вызвать SAT-решатель для C

    если результат SAT:
        добавить K_i в Core
        сохранить возвращённый набор α_i

    если результат UNSAT:
        K_i в Core не включать

вернуть Core

```

Сохранённый набор α_i — свидетель ядерности K_i : по нему можно непосредственно проверить

$$K_i(\alpha_i) = 1, \quad K_j(\alpha_i) = 0 \quad (j \neq i).$$

При проверке каждого терма всегда используются все остальные термы исходной сокращённой ДНФ: уже признанные неядерными термы из последующих запросов удалять нельзя.

5. Пример

Рассмотрим сокращённую ДНФ

$$A_f = x_1x_2 \vee \overline{x_1}x_3 \vee x_2x_3$$

со всеми простыми импликантами

$$K_1 = x_1x_2, \quad K_2 = \overline{x_1}x_3, \quad K_3 = x_2x_3.$$

Проверка K_1 . Строим

$$F_1 = x_1 \wedge x_2 \wedge (x_1 \vee \overline{x_3}) \wedge (\overline{x_2} \vee \overline{x_3}).$$

Единичные дизъюнкты дают $x_1 = 1$, $x_2 = 1$; тогда последний дизъюнкт требует $x_3 = 0$. Выполняющий набор:

$$\alpha_1 = (1, 1, 0).$$

Значит, K_1 — ядерная импликанта.

Проверка K_2 . Строим

$$F_2 = \overline{x_1} \wedge x_3 \wedge (\overline{x_1} \vee \overline{x_2}) \wedge (\overline{x_2} \vee \overline{x_3}).$$

Единичное распространение даёт

$$x_1 = 0, \quad x_3 = 1, \quad x_2 = 0.$$

Набор $\alpha_2 = (0, 0, 1)$ выполняет F_2 , поэтому K_2 тоже ядерная.

Проверка K_3 . Строим

$$F_3 = x_2 \wedge x_3 \wedge (\overline{x_1} \vee \overline{x_2}) \wedge (x_1 \vee \overline{x_3}).$$

Из $x_2 = 1$, $x_3 = 1$ два последних дизъюнкта превращаются соответственно в

$$\overline{x_1}, \quad x_1,$$

что противоречиво. Следовательно, F_3 невыполнима, и K_3 не является ядерной.

Итог:

$$\bigcap T_f = \{x_1x_2, \overline{x_1}x_3\}.$$

6. Доказательство корректности

Если алгоритм включает K_i в Core, то решатель нашёл выполняющий набор α формулы F_i . По построению F_i :

$$K_i(\alpha) = 1, \quad K_j(\alpha) = 0 \quad (j \neq i).$$

Так как вход содержит все простые импликанты, получаем

$$\Pi(f, \alpha) = \{K_i\},$$

то есть K_i — ядерная импликанта, $K_i \in \bigcap T_f$.

Обратно, если $K_i \in \bigcap T_f$, то по теореме из лекций существует набор α с

$$\Pi(f, \alpha) = \{K_i\}.$$

Этот набор выполняет F_i , поэтому полная SAT-процедура обязана признать F_i выполнимой и включить K_i в результат.

Следовательно, алгоритм возвращает в точности

$$\text{Core} = \bigcap T_f.$$

7. Оценка объёма вычислений

Пусть K_i содержит r_i литералов ($r_i \leq n$). Каждый запрос содержит:

- не более $n = 100$ переменных;
- r_i единичных дизъюнктов;
- $m - 1 = 999$ дизъюнктов — отрицаний остальных термов.

Итого не более

$$100 + 999 = 1099$$

дизъюнктов на запрос; всего требуется $m = 1000$ SAT-запросов.

Если суммарное число вхождений литералов равно

$$L = \sum_{j=1}^m r_j,$$

то каждый запрос содержит ровно L вхождений литералов, построение одного запроса — $O(L)$, всех запросов — $O(mL)$. Время решения зависит от сложности конкретных SAT-экземпляров.

Метод не строит таблицу истинности из 2^{100} строк, а проверяет существование «единственного» набора посредством единичного распространения, ветвления и обработки конфликтов.